

Как мы пришли к submission

submission_v10_conditional_disagree.zip

с результатом 12.66

1. Исходная задача

Нужно было предсказать цену автомобиля по табличным признакам. В распоряжении были:

- обучающая выборка: 8 340 объектов с целевой переменной Цена ;
- тестовая выборка: 8 341 объект без целевой переменной;
- конкурсная метрика — MAPE :

$$\text{MAPE} = \text{mean}(\text{abs}(y_{\text{true}} - y_{\text{pred}}) / y_{\text{true}}) * 100$$

То есть ошибка измеряется в процентах от настоящей цены. Ошибка на машине стоимостью 10 000 и прогнозе 12 000 равна 20%, как и на машине стоимостью 100 000 при прогнозе 120 000.

Это важно: модели нельзя было оценивать только по обычному RMSE в долларах, потому что крупные дорогие машины иначе чрезмерно влияли бы на качество.

2. Сначала привели данные к безопасному виду

2.1. Исключили car_id

car_id — технический идентификатор строки. Он не несёт осмысленной информации о цене и может приводить к случайным ложным закономерностям.

Поэтому он не использовался как признак модели.

2.2. Нашли потенциальную утечку — Предложение

В обучающей выборке признак Предложение почти напрямую связан с ценой: корреляция была около 0.994 .

Но в тесте этот признак отсутствовал или был неполным. Использовать его означало бы построить модель на информации, которой нет в финальной задаче. Такая модель могла бы показать красивый результат на train, но не переносилась бы на test.

Поэтому Предложение был окончательно исключён.

2.3. Преобразовали строковые технические характеристики

Некоторые числовые поля были записаны строками. Из них были извлечены нормальные числовые признаки:

Пробег	→ Пробег_число
Расход	→ Расход_л_на_100км
Количество цилиндров	→ Двигатель_цилиндры
Двери	→ Двери_число
Количество кресел	→ Кресла_число
Двигатель	→ Двигатель_объём_л

Дополнительно были нормализованы пропуски и текстовые категории.

3. Построили честную систему валидации

Основой стала 5-fold OOF-валидация.

Идея:

1. Делим train на 5 частей.
2. На четырёх частях обучаем модель.
3. На пятой получаем прогноз.
4. Повторяем для всех пяти частей.
5. В итоге каждая строка train получает прогноз от модели, которая эту строку не видела во время обучения.

Это называется `out-of-fold prediction`, или OOF.

Для сохранения распределения цен folds строились со стратификацией по квантилям таргета:

```
StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42,
)
```

Это позволило:

- честно сравнивать модели;
- строить ансамбли;
- проверять calibration;
- не выбирать веса по прогнозам, обученным на тех же объектах.

Для особенно чувствительных этапов использовалась nested-логика:

- на части фолдов подбирались веса и множители;
- на отложенном внешнем фолде проверялось, переносится ли улучшение.

4. Первые базовые модели

Мы не искали одну «идеальную» модель. Цель была другой: собрать несколько моделей, которые ошибаются по-разному, а затем объединить их.

4.1. Ridge на логарифме цены

Линейная модель на логарифме цены дала OOF MAPE:

14.468951

Она была заметно слабее CatBoost, но оказалась полезной в ансамбле, потому что давала иной тип ошибок.

4.2. Text Ridge по текстовому описанию

Была построена отдельная модель на текстовых признаках и названии автомобиля через TF-IDF + Ridge.

Её самостоятельный OOF MAPE:

14.903406

Отдельно модель слабая, но в ансамбле она сохранила небольшой положительный вес, потому что текст улавливает редкие слова, комплектации и маркетинговые формулировки, которые другие модели могут использовать иначе.

5. Последовательное развитие CatBoost-моделей

5.1. V4: CatBoost с названием и базовыми признаками

Первая сильная CatBoost-модель с обработанными характеристиками и названием автомобиля дала:

OOF MAPE: 13.050142

5.2. V5: иерархия признаков из названия

Название автомобиля содержало много полезной информации: марка, семейство, версия, двигатель, трансмиссия, комплектация, привод.

Мы начали извлекать из него структуру:

Название_без_года
Название_нормализованное_без_года
Название_префикс_2
Название_префикс_3
Название_префикс_4
Название_мощность_kw
Название_есть_мощность_kw
Название_моторный_маркер
Название_число_слов

Также были добавлены флаги, например:

4X4, AWD, TURBO, SPORT, HYBRID, GT, LUXURY,
DIESEL, AMG, M_SPORT, RS, S_LINE, GTI,
HSE, SR5, GXL, LIMITED, PREMIUM,
COMFORTLINE, ASCENT, ACTIVE, ELITE,
TDI, TSI, TFSI, CDI, V6, V8

Результат V5:

OOF MAPE: 12.755321

5.3. V6 и V7: убрали шумные признаки

Мы проверили, что не все исходные текстовые поля помогают.

В частности, отдельные варианты с полным названием и цветом не давали устойчивого выигрыша. После чистки признаков получилось:

V6 OOF MAPE: 12.746942
V7 OOF MAPE: 12.681645

Главный вывод: не нужно держать в модели все доступные признаки. Иногда удаление нестабильного или шумного признака улучшает перенос на test.

5.4. V8: глубокие признаки комплектации из названия

Дальше название было разобрано точнее. Добавились признаки:

Название_комплектация_1
Название_комплектация_2
Название_комплектация_3
Название_первые_2

Название_первые_3
Название_число_токенов

И дополнительные индикаторы:

QUATTRO, XDRIVE, 4MATIC, ALLGRIP, ALLTRACK, EQUIPPED

V8 стала основной сильной моделью:

OOF MAPE: 12.446801

Это был заметный шаг вперёд относительно V7.

6. Fold-safe target statistics

Обычный CatBoost хорошо использует признаки, но некоторые связи особенно важны для автомобильного рынка:

марка + модель
марка + модель + год
нормализованное название
нормализованное название + год
префикс названия
префикс названия + год

Например, цена Toyota Camry 2019 года принципиально отличается от цены Toyota Camry 2009 года, даже если часть остальных характеристик похожа.

Для этого были построены target statistics:

- количество наблюдений в группе;
- сглаженное среднее логарифма цены.

Ключевой момент: статистики создавались строго cross-fitted.

То есть для каждой строки train средняя цена её группы считалась только по другим фолдам, без использования её собственного таргета. Это защищало от target leakage.

Самостоятельная stats-модель дала:

OOF MAPE: 12.599081

Она была сильной сама по себе и особенно полезной как независимый компонент ансамбля.

7. Retrieval: поиск похожих автомобилей

Следующий сильный слой — retrieval, то есть поиск аналогов.

Для автомобиля искали похожие объекты прежде всего внутри пары:

Бренд + Модель

Если таких объектов не было — использовался fallback:

Бренд → весь train

Сходство считалось по:

Год выпуска
Пробег
Объём двигателя
Число цилиндров
Двери
Кресла
Топливо
КПП
Привод
Тип кузова
Штат
Комплектация 1
Комплектация 2

Для ближайших трёх аналогов строилась взвешенная оценка цены. Более близкие автомобили получали больший вес.

Сама retrieval-модель была слабой:

OOF MAPE: около 16.2

Но это не означало, что она бесполезна.

Она ошибалась иначе, чем CatBoost. Поэтому добавление retrieval к V8 заметно улучшило смесь моделей:

V8 + retrieval: OOF MAPE около 12.10

Главный ML-вывод здесь такой:

Модель может быть слабой сама по себе, но сильной в ансамбле, если она добавляет независимую информацию.

8. Первый сильный ансамбль

В итоговый ансамбль вошли:

```
Ridge
V5
V6
V8
target-stats CatBoost
text Ridge
retrieval
```

Весы не задавались вручную. Они подбирались оптимизацией непосредственно под MAPE с ограничениями:

```
веса >= 0
сумма весов = 1
```

После появления retrieval nested OOF качество ансамбля было примерно:

```
11.730500
```

На leaderboard этот этап дал улучшение примерно до:

```
12.89
```

9. V10: объединение V8 и fold-safe target statistics

V10 объединила два сильных источника информации:

```
V8-признаки из названия и характеристик
+
fold-safe target statistics
```

Получилось 81 признаков поле:

69 признаков V8
+
12 target-statistics признаков

На holdout V10 сама по себе не обогнала V8, но смесь V8 и V10 дала заметный выигрыш. Это означало, что V10 опять же полезна не потому, что она обязательно лучше отдельно, а потому что она иначе ошибается.

Сначала был сделан проху-подход: V10 частично заменяла блок V8 + stats в финальном предсказании.

Этот проху дал реальный leaderboard-результат:

12.80

После подтверждения на leaderboard была проведена полная 5-fold OOF-проверка V10.

Полный ансамбль с V10 показал:

Direct OOF: 11.692472
Nested OOF: 11.715950

V10 получила в финальном ансамбле вес около:

0.161980

10. Что мы проверили и сознательно закрыли

Важно, что улучшение строилось не только на удачных идеях. Мы также быстро закрывали направления, которые не переносились.

10.1. Дополнительные target-statistics группы

Были проверены более узкие группировки по топливу и коробке передач. На одном seed они выглядели лучше, на другом — хуже.

Вывод: эффект нестабилен, ветка закрыта.

10.2. Adversarial validation

Проверили, отличаются ли train и test системно.

AUC adversarial validation был около:

0.502467

Это почти случайный уровень.

Вывод: заметного train-test shift нет, сложное reweighting по доменам не требуется.

10.3. Взвешивание дешёвых автомобилей

Проверялась идея сильнее оптимизировать нижний ценовой сегмент. Она не дала устойчивого улучшения.

Ветка закрыта.

10.4. Dynamic retrieval

Проверялась идея давать retrieval разный вес в зависимости от качества соседей: расстояния, разброса цен аналогов, совпадения комплектаций.

Эта схема не превзошла фиксированный retrieval-слой.

Ветка закрыта.

10.5. Residual retrieval

Следующая идея была: не брать цену аналогов напрямую, а использовать их локальные остатки:

```
итоговый прогноз
=
прогноз ансамбля
+
локальная ошибка похожих объектов
```

Но связь между предсказанным локальным остатком и реальной ошибкой была почти нулевая:

```
Pearson:  -0.011960
Spearman: -0.022811
```

Коррекция не дала улучшения.

Ветка закрыта.

10.6. Прямая оптимизация CatBoost MAPE

Проверили CatBoost с `loss_function="MAPE"` на исходной цене.

Результат оказался резко хуже:

V8 на holdout:	12.488853
CatBoost direct MAPE:	30.177999

Модель системно занижала цены и особенно плохо работала на дорогих автомобилях.

Вывод:

RMSE на $\log_{10}(\text{Цена})$
+
ансамбль
+
отдельная calibration

для этой задачи намного лучше прямой оптимизации MAPE внутри CatBoost.

10.7. Плавный spline-Ridge calibrator

После удачной conditional calibration проверили более сложный калибратор, который пытался учить поправку по:

разбросу моделей
цене
году
пробегу
двигателю
конфликту retrieval с основными моделями
разнице V8 и V10

Но он проиграл простой трёхкорзинной схеме:

3-bin calibration:	11.472224
smooth spline-Ridge:	11.671761

Это хороший пример того, что более сложная модель не всегда лучше. Простая схема оказалась устойчивее.

11. Главный прорыв: conditional calibration по disagreement моделей

Мы заметили, что разброс прогнозов моделей связан с ошибкой итогового ансамбля.

Использовался сигнал:

```
relative_std = std(predictions_of_8_models) / mean(predictions_of_8_models)
```

То есть для каждого автомобиля мы смотрели, насколько между собой согласны:

```
Ridge
V5
V6
V8
stats model
text model
retrieval
V10
```

Связь между disagreement и ошибкой оказалась сильной:

```
Spearman correlation: 0.372798
```

Это означает:

- если модели близки по прогнозу, объект типичный и предсказуемый;
- если модели сильно расходятся, автомобиль редкий, дорогой, нестандартный или плохо покрыт данными;
- именно в зоне сильного disagreement ансамбль чаще завышал цену.

11.1. Глобальная calibration

До этого использовался единый множитель для всех машин:

```
примерно ×0.98
```

Она улучшала nested OOF:

```
raw exact ensemble: 11.715944
global calibration: 11.572301
```

11.2. Calibration по трём корзинам disagreement

Все автомобили были разделены на три равные группы по относительному разбросу моделей.

Получились устойчивые множители:

низкий disagreement: ×0.994
средний disagreement: ×0.980
высокий disagreement: ×0.951

Логика:

модели согласны → почти не корректируем прогноз;
модели немного расходятся → стандартное снижение;
модели сильно расходятся → сильнее снижаем прогноз.

Nested OOF результат:

global calibration: 11.572301
conditional disagreement calibration: 11.472216

Улучшение:

-0.100085 п.п.

И главное: conditional calibration выиграла глобальную на всех пяти внешних фолдах.

12. Финальная модель

Финальный submission состоит из взвешенного ансамбля восьми компонентов.

Веса ансамбля

V8	0.348682
Ridge	0.196562
V10	0.161980
retrieval K=3	0.132080
target-stats model	0.101857
text Ridge	0.038848
V5	0.019991
V6	0.000000

V6 осталась в таблице для честности оптимизации, но её финальный вес стал нулевым.

Финальная формула

```
raw_prediction =  
    0.348682 × V8  
    + 0.196562 × Ridge
```

```
+ 0.161980 × V10
+ 0.132080 × Retrieval
+ 0.101857 × Stats
+ 0.038848 × Text Ridge
+ 0.019991 × V5
```

После этого вычислялся disagreement всех восьми прогнозов.

Затем применялся один из трёх множителей:

```
низкий disagreement → ×0.994
средний disagreement → ×0.980
высокий disagreement → ×0.951
```

Проверка распределения test

Доли test-объектов по корзинам были очень близки к OOF:

```
низкий disagreement: 35.020%
средний disagreement: 33.114%
высокий disagreement: 31.867%
```

Это важный аргумент в пользу переноса calibration: test по структуре не оказался сильно отличающимся от train.

13. Итоговый результат

Финальный submission:

```
submission_v10_conditional_disagree.zip
```

Результат на leaderboard:

```
12.66
```

Динамика лучших отправок:

```
V8 + retrieval: 12.89
V10 proxy: 12.80
V10 + conditional calibration: 12.66
```

Последнее улучшение:

12.80 → 12.66
-0.14 п.п.

14. Главные выводы проекта

1. Нельзя ориентироваться только на качество отдельной модели

Retrieval и text Ridge были слабее V8 сами по себе, но улучшили ансамбль, потому что давали другой взгляд на цену автомобиля.

2. OOF и nested validation критичны

Без них можно легко получить «улучшение» за счёт подгонки весов, target statistics или calibration на тех же объектах.

3. Target encoding допустим только при cross-fitting

Иначе это скрытая утечка таргета.

4. Простая calibration может быть сильнее сложной meta-модели

Трёхкорзинная схема победила spline-Ridge calibrator, потому что её закономерность была простой, понятной и устойчивой.

5. Disagreement моделей — полезный признак неопределённости

Расхождение независимых моделей помогло не только оценить риск ошибки, но и исправить систематическое завышение прогноза для сложных объектов.

6. Реальный leaderboard важнее красивого локального результата

Каждый новый рискованный подход сначала проверялся по OOF, а удачные гипотезы затем подтверждались реальным submission.

Именно поэтому итоговый результат 12.66 — не случайный набор эвристик, а последовательная композиция проверенных сигналов.